Model training - evidence pack

Certified anytime perception under post-quantum cryptographic load
Generated 2026-10-09 from the saved training artifacts.

0. Scope: what is trained and what is not

One component was trained with gradient descent; the rest are a statistical fit, a simulator over measured costs, or measurements. Stating this precisely is part of the result.

Component	What it actually is	Gradient-trained?
Multi-exit perception model	3-exit MobileNetV1, Keras, 100 epochs	**Yes** - train/train.py
Baseline control model	Same backbone, single final exit	**Yes** - train/train.py
Learned exit policy (conformal)	Isotonic / PAVA risk curves fitted on the calibration split	No - statistical fit
Joint PQC + exit scheduler	Trace-driven simulator over measured costs	No
ML-DSA latency tails	Measured distributions, 2000 runs per level	No

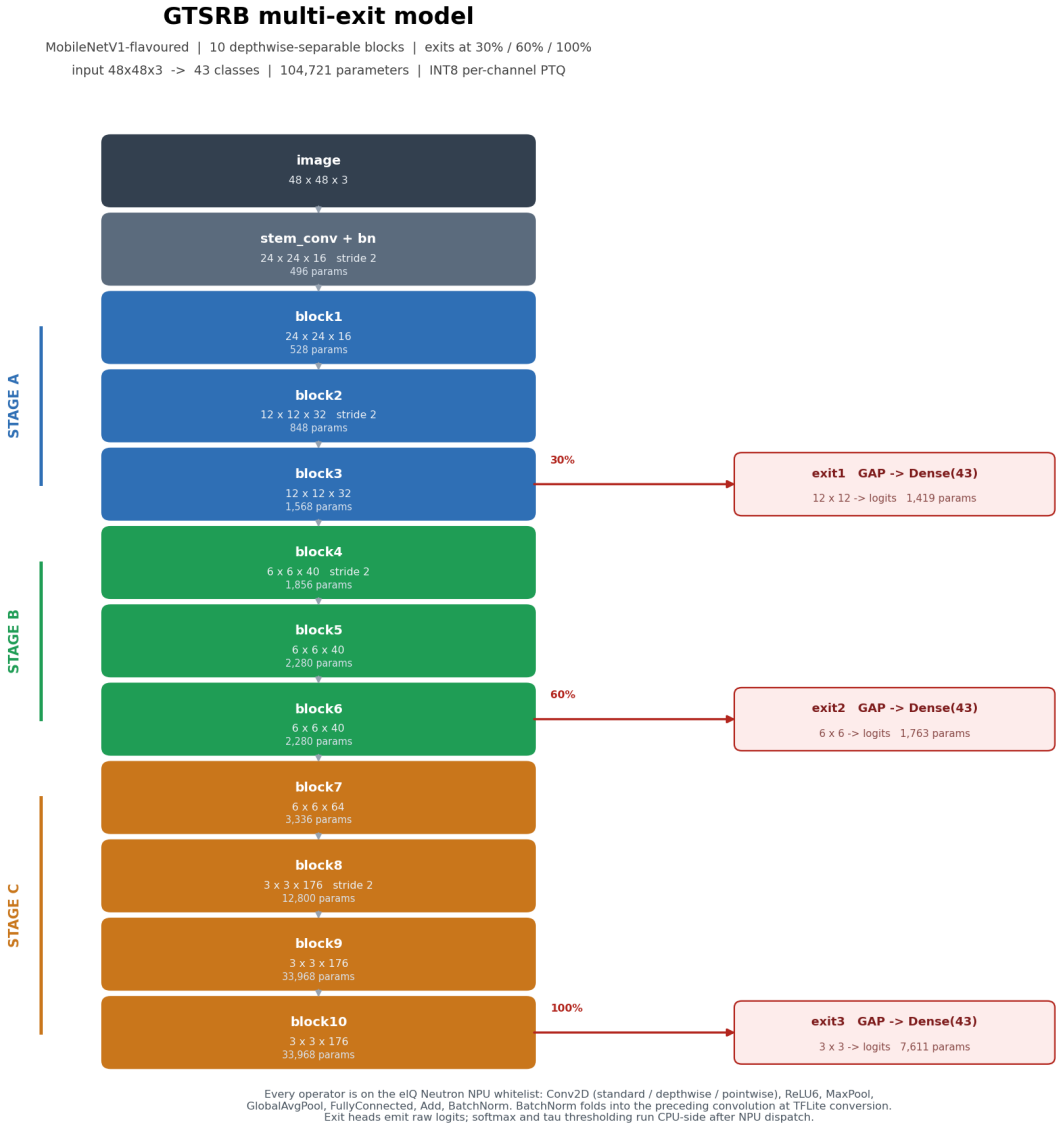
There is no jointly-trained PQC + perception neural network. Only two scripts in the tree perform gradient training (train.py, quantize.py), and both train the perception model. The layer that decides which security level and which exit to use is analytical: a simulator driven by real measured costs, which is the stronger choice when every cost in it is a measurement rather than a learned proxy.

1. The trained model

Property	Value
Input	48 x 48 x 3, INT8-quantised
Classes	43 (GTSRB)
Depth	10 depthwise-separable blocks in 3 stages
Exit placement	**exactly 30% / 60% / 100%** of block depth (blocks 3, 6, 10)
Stage widths	stem 16 -> A 32 -> B 40 -> C 176
Operators	Conv2D (std / depthwise / pointwise), ReLU6, MaxPool, GlobalAvgPool, FC, Add, BatchNorm
Excluded	SE blocks, attention, swish, in-graph softmax
Exit heads	raw logits; softmax + tau thresholding applied CPU-side after NPU dispatch

Capacity is deliberately unequal across exits. Stage B was narrowed from 64 to 40 channels and stage C widened from 128 to 176 *because* at 64 channels exits 2 and 3 converged to within about 1 percentage point by epoch 15 and stayed there - the exit-collapse failure mode, for the exit2/exit3 pair. Head placement is exact rather than approximate: blocks 3, 6 and 10 of 10 give 30 / 60 / 100%.

1b. Architecture



One box per depthwise-separable block. Each block is depthwise conv + BatchNorm + pointwise conv + BatchNorm. Exits tap the trunk after blocks 3, 6 and 10 (30% / 60% / 100%). Total 104,721 parameters, verified against the checkpoint.

2. Training configuration

```
train/configs/multiexit.yaml
```

```
seed: 20260728
model: multiexit
batch_size: 128
epochs: 100
learning_rate: 0.001
val_track_fraction: 0.1
exit_loss_weights: {exit1: 1.0, exit2: 1.0, exit3: 1.0}
```

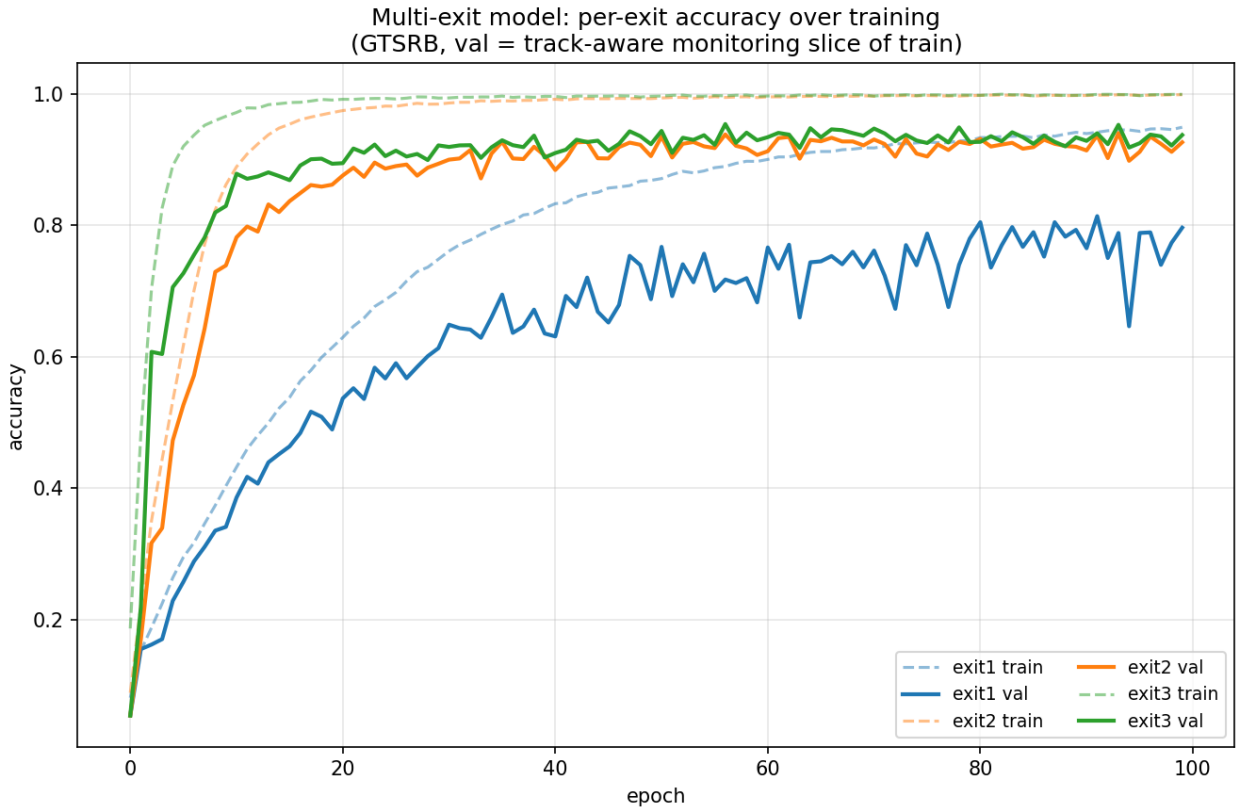
Loss. BranchyNet joint loss, L = sum_k w_k * CE(exit_k), equal weights w_k = 1.0. Equal weighting follows the brief; per-exit weighting is the documented lever if the accuracy spread needs rebalancing.

Validation split. Track-aware and taken from the training set only. GTSRB frames are named <trackID>_<frameID>.ppm with roughly 30 near-duplicate frames per physical sign, so a random split would leak the same sign into train and validation. val_track_fraction 0.1 holds out whole tracks. Calibration and test splits are untouched by training.

Baseline control. train/configs/baseline.yaml: identical backbone, single final exit, single cross-entropy. Used to measure what joint multi-exit training costs the final classifier.

3. Training curves

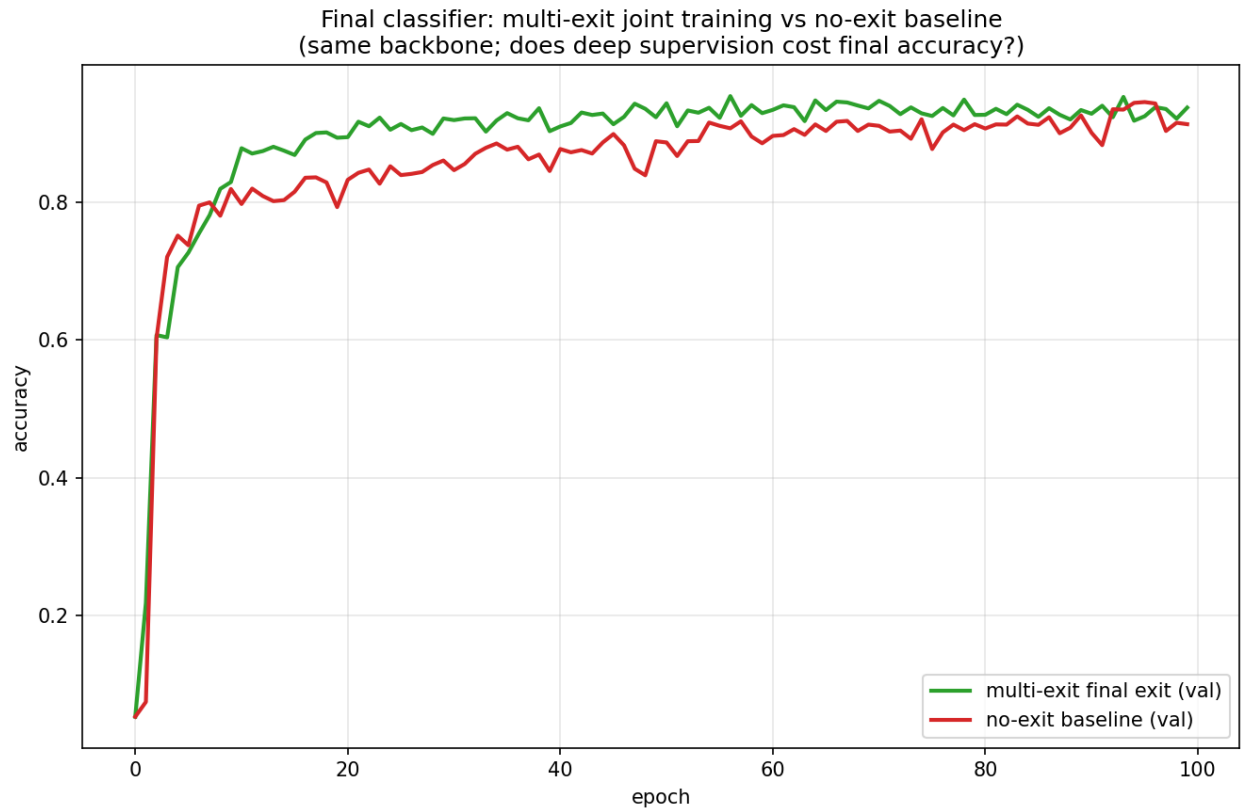
Source: results/logs/{multiexit,baseline}/history.csv, one row per epoch. Regenerate with `python make_plots.py`.



Per-exit training and validation accuracy.



Per-exit training and validation loss.



Multi-exit joint training vs the single-exit baseline control.

4. Key numbers

Read from the history files at build time.

Metric	Epoch 0	Final (100)	Best	Best epoch
val exit1 accuracy	0.048	0.727	**0.733**	95
val exit2 accuracy	0.048	0.901	**0.923**	94
val exit3 accuracy	0.048	0.909	**0.931**	94
val loss (summed over exits)	10.918	2.093	**1.770**	94

Baseline control	Epoch 0	Final (100)	Best	Best epoch
val accuracy	0.054	0.913	**0.945**	95
val loss	3.825	0.498	**0.360**	96

What the curves show

1. All three exits learn and stay ordered: exit3 (0.931) > exit2 (0.923) > exit1 (0.733). The ordering is what makes the anytime behaviour meaningful - if the exits collapsed together there would be nothing to trade off.

2. The spread is wide and stable - about 20 points between exit1 and exit3. That is the deliberate capacity handicap working.

3. Joint multi-exit training costs the final exit about 1.4 points (0.931 vs 0.945 for the baseline). That is the price of the anytime property, and it is the number to quote when asked whether early exits are free.

4. Both runs peak near epoch 94-96 and drift slightly afterwards; training to 100 epochs is past the optimum.

Checkpoint selection

best_joint.weights.h5 is chosen by a MaxOfMinCheckpoint callback that maximises min(val_exit1_acc, val_exit2_acc, val_exit3_acc) - it protects the weakest exit rather than the average. On this run that lands at **epoch 95** (min-of-exits = 0.733, set by exit1).

Available: best.weights.h5, best_exit3_epoch71.weights.h5, best_exit3_frozen_epoch_NA.weights.h5, best_joint.weights.h5, final.weights.h5.

5. From trained weights to device segments

Step	Script	Output
Train	train/train.py	results/checkpoints/multiexit/*.weights.h5
Quantise (INT8 per-channel PTQ)	train/quantize.py	results/tflite/exit{1,2,3}_int8.tflite
Split into standalone per-exit segments	-	results/segments/seg{A,B,C}_int8.tflite
Quantisation accuracy check	-	results/quantize_accuracy_report.csv

Segment sizes: segA 16,736 B, segB 21,168 B, segC 125,896 B. All three share one trained graph; they are separate .tflite files so each can run standalone on device.

6. Components that are not gradient-trained

Learned exit policy. conformal/learned_exit_policy.py fits isotonic (PAVA, decreasing) risk curves risk_k(c) = P(safety-critical AND wrong | conf_k = c) on the calibration split only, then selects the shallowest exit whose risk is under the target alpha. It is a monotone curve fit with a guarantee argument, not a trained network. Output: results/learned_policy/risk_curves.json.

Joint scheduler. conformal/joint_scheduler.py is a trace-driven simulator. State per frame: conf1, conf2, cryptoImminent, budgetUsed. Decision per frame: which exit. Decision per handshake: which ML-KEM level (512 / 768 / 1024). All costs are measured:

```
exit1 = 0.80 ms, exit2 = 1.24 ms, exit3 = 2.34 ms (MCXN947, on device)
ML-KEM handshake: 512 = 72.17 ms, 768 = 115.97 ms, 1024 = 174.91 ms
```

Correction to apply before quoting the scheduler. Those ML-KEM figures are the bare-metal PQClean numbers. The later -O2 wolfSSL harness measured the same ML-KEM-768 handshake at **21.62 ms** - 5.4x faster. The scheduler's "infeasible at level 768" conclusion flips to feasible at the real cost, so the report should be re-run against the faster numbers before it is presented.

Conformal certification itself is not learned: it is the split-conformal quantile calculation over the calibration split, reported in results/conformal_tau_table.csv and results/coverage_validation_table.csv.

7. Files

Purpose	Path
Raw per-epoch history	training/results/logs/multiexit/history.csv
Baseline history	training/results/logs/baseline/history.csv
Config	training/train/configs/multiexit.yaml
Model definition	training/models/architectures.py
Checkpoints	training/results/checkpoints/multiexit/
Certification results	training/results/conformal_tau_table.csv
Plot regeneration	python make_plots.py (in training/)